Object Oriented Programming (CSEG1044)
Unit 04 – Lecture 05 Notes
Swing Event Handling + JDBC (Persistence)

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Swing event model (listeners) . . . 3

7.2 JDBC core flow (pattern you should memorize) . . . 4

7.3 `Statement` vs `PreparedStatement` vs `CallableStatement` . . . 4

7.4 `ResultSet` traversal . . . 4

7.5 Persistent data design (UI vs DAO) . . . 5

8 Running Project Connection **5**

9 Common Mistakes and Confusions **5**

10 Worked Example: Save a resource from Swing to DB (skeleton) **5**

11 Demo / Observation Task **7**

12 Practice Check (with brief solutions) **7**

13 Self-Study Practice (no solutions) **8**

14 Lab Alignment (recommended) **8**

15 Further Study **8**

16 Exit Questions (with sample answers) **8**

1 Lecture Snapshot

- **Unit focus:** Generics, Lambdas, Swing, and JDBC
- **Main new ideas:** Swing event model (listeners), JDBC core flow (pattern you should memorize), `Statement` vs `PreparedStatement` vs `CallableStatement`
- **Course thread:** Use Swing Event Handling + JDBC (Persistence) to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

2 Lecture Overview

In the last lecture we built a Swing UI skeleton. Now we make it interactive and persistent:

- **Events**: a button click triggers code (save/clear/exit).
- **Persistence**: data is stored in a database using JDBC so it is not lost after closing the app.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Swing foundations**: You already know the component structure; now user actions and persistence are attached to that UI. Main reference: `Unit 04 / Lecture 04 slides/notes`.
- **Failure handling**: JDBC and event code both need disciplined exception handling and user messages. Main reference: `Unit 03 / Lecture 02 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Connect Swing UI controls to actions using listeners.
- Explain JDBC flow and choose between `Statement` and `PreparedStatement`.
- Implement basic CRUD and read results using `ResultSet`.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Swing event model (listeners) → JDBC core flow (pattern you should memorize) → `Statement` vs `PreparedStatement` vs `CallableStatement` → `ResultSet` traversal → Persistent data design (UI vs DAO).*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Event source**: UI component that triggers an event (button, menu item).
- **Event object**: information about the event (e.g., `ActionEvent`).
- **Listener**: object/function that handles events (e.g., `ActionListener`).
- **EDT**: Event Dispatch Thread (Swing UI thread).
- **JDBC**: Java Database Connectivity API.
- **Connection**: active DB connection.
- **PreparedStatement**: parameterized SQL statement (safer and common).
- **ResultSet**: table-like cursor over query results.
- **CRUD**: Create, Read, Update, Delete.
- **DAO**: Data Access Object (separate DB code from UI).

7 Detailed Notes

7.1 Swing event model (listeners)

Swing uses a listener-based model:

- You create a component (e.g., `JButton saveBtn`).
- You register a listener: `saveBtn.addActionListener(...)`.
- When user clicks, Swing calls your listener method on the EDT.

Common controls and events:

- `JButton` → `ActionListener`
- `JTextField` → `ActionListener` (press Enter), `DocumentListener` (text changes)
- `JTable` → selection listeners (advanced)

Important rule. Do not run slow DB operations on the EDT; it can freeze the UI. In a basic course project, keep operations small or use a background thread/SwingWorker (advanced).

7.2 JDBC core flow (pattern you should memorize)

1. Load/ensure driver is available (often automatic in modern JDBC).
2. Get a connection: `DriverManager.getConnection(url, user, pass)`.
3. Create a statement (usually `PreparedStatement`).
4. Execute:
   - `executeUpdate()` for INSERT/UPDATE/DELETE
   - `executeQuery()` for SELECT
5. Read results using `ResultSet` (cursor).
6. Close resources (use `try-with-resources`).

7.3 `Statement` vs `PreparedStatement` vs `CallableStatement`

- `Statement`: simplest; builds SQL as a string (risk of SQL injection; avoid for user input).
- `PreparedStatement`: parameterized with `?`; safer and faster for repeated queries.
- `CallableStatement`: calls stored procedures (depends on DB).

7.4 `ResultSet` traversal

- `while (rs.next()) { ... }` moves row by row.
- Read columns by label or index: `rs.getInt("id")`, `rs.getString("name")`.
- Map each row to an object (e.g., `Resource`).

7.5 Persistent data design (UI vs DAO)

Avoid writing SQL inside button click code. Preferred structure:

- UI layer: reads fields, validates, shows messages.
- DAO layer: contains SQL and JDBC code.
- Model layer: `Resource` class that represents one row/entity.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

8 Running Project Connection

Use Swing Event Handling + JDBC (Persistence) to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Building SQL strings directly from user input.
- Running slow database work on the Event Dispatch Thread.
- Mixing UI controls, validation, and DAO logic in one method.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

10 Worked Example: Save a resource from Swing to DB (skeleton)

1) Model class

```
public class Resource {
    private final String name;
    private final String type;
```

```java
    private final int capacity;

    public Resource(String name, String type, int capacity) {
        this.name = name;
        this.type = type;
        this.capacity = capacity;
    }

    public String getName() { return name; }
    public String getType() { return type; }
    public int getCapacity() { return capacity; }
}
```

2) DAO (insert + select)

```java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ResourceDao {
    private final String url = "jdbc:mysql://localhost:3306/crm"; // example
    private final String user = "root";
    private final String pass = "password";

    public void insert(Resource r) throws SQLException {
        String sql = "INSERT INTO resources(name, type, capacity) VALUES(?,?,?)";
        try (Connection con = DriverManager.getConnection(url, user, pass);
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, r.getName());
            ps.setString(2, r.getType());
            ps.setInt(3, r.getCapacity());
            ps.executeUpdate();
        }
    }

    public List<Resource> findAll() throws SQLException {
        String sql = "SELECT name, type, capacity FROM resources";
        List<Resource> out = new ArrayList<>();
        try (Connection con = DriverManager.getConnection(url, user, pass);
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                out.add(new Resource(
                    rs.getString("name"),
                    rs.getString("type"),
                    rs.getInt("capacity")
                ));
            }
        }
        return out;
    }
}
```

3) Event handler (button click)

```
saveBtn.addActionListener(e -> {
    String name = nameField.getText().trim();
    String type = typeField.getText().trim();
    String capText = capacityField.getText().trim();

    if (name.isEmpty() || type.isEmpty() || capText.isEmpty()) {
        JOptionPane.showMessageDialog(f, "All fields required");
        return;
    }

    int cap;
    try {
        cap = Integer.parseInt(capText);
        if (cap <= 0) throw new NumberFormatException();
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(f, "Capacity must be a positive integer");
        return;
    }

    saveInBackground(name, type, cap);
});
```

The listener validates input and delegates the JDBC work. The DB call should run in a background task. `SwingWorker` is a simple option that keeps the EDT responsive.

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Swing event model (listeners)* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Which is safer for user input: `Statement` or `PreparedStatement`?

   Answer: `PreparedStatement`.

2. Why should heavy DB operations not run on the EDT?

Answer: the UI can freeze because Swing processes events on the EDT.

3. What method is used for SELECT queries: `executeQuery()` or `executeUpdate()`?

   Answer: `executeQuery()`.

13 Self-Study Practice (no solutions)

1. Add an `updateCapacity(name, newCapacity)` method in `ResourceDao`.
2. Add a "Load All" button that displays all resources in a text area/table.
3. Replace DB credentials with a config file and load them at startup (optional).

14 Lab Alignment (recommended)

- Exp-16: GUI using Swing
- Exp-17: Database connectivity using JDBC

15 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

16 Exit Questions (with sample answers)

Q1. Why is `PreparedStatement` preferred?

Sample answer: It allows parameterized queries, prevents SQL injection for user input, and can be more efficient for repeated execution.

Q2. What is the role of a DAO?

Sample answer: A DAO separates database code from UI/business logic, making the application easier to maintain, test, and change.

Q3. Why is PreparedStatement the default choice for CRUD forms?

Sample answer: It supports parameterized queries and makes database operations safer and cleaner.